

Tracing Stylistic Change in Billboard Pop Lyrics, 1965–2025

A Multi-Feature Text-Mining Analysis with a Recommendation Agent

1. Overview

This project applies text mining to **4,869 Billboard Year-End Hot 100 songs spanning 1965–2025** to ask two questions:

1. **Have lyrical styles measurably changed across decades?** We answer with seven feature families (TF-IDF, rhyme, repetition, NRC emotion, Brysbaert concreteness, pronouns, SBERT embeddings) fed to a logistic-regression decade classifier with proper cross-validation.
2. **Can the same feature pipeline power something useful?** We build a recommendation agent that takes a user's lyrical taste profile (z-scored sliders) and uses OpenAI's tool-use API to search Genius, fetch live lyrics for candidate recent songs, score them on the same features, and rank by cosine similarity.

The project anchors itself in Parada-Cabaleiro et al. 2024 (*Scientific Reports*), who used a similar feature stack on a five-decade, five-genre corpus and found lyrics have become "simpler and more repetitive." We replicate a slice of those findings on Billboard-only data and extend them with a working interactive artifact.

2. Dataset

Source	Coverage	Songs
walkerkq/musiclyrics (GitHub)	Billboard Year-End Hot 100, 1965–2015	4,633
Genius API supplemental scrape	Billboard Hot 100 chart-toppers, 2016–2025	236
Total	1965–2025	4,869

Decade distribution (skewed by walkerkq's pre-2015 focus + our smaller recent scrape):

Decade	Songs
1960s	456
1970s	910
1980s	952
1990s	886
2000s	898
2010s	627
2020s	140

Important data caveat: walkerkq stored lyrics as a single concatenated string with line breaks stripped. This means line-based features (rhyme density, internal rhyme, syllables-per-line, chorus repetition ratio) are biased toward zero for 1965–2015 songs and become meaningful only for the 2016–2025 subset scraped fresh from

Genius. Bag-of-words features (emotion, concreteness, pronouns, lexical diversity, SBERT embeddings) are unaffected because they don't depend on line structure. This caveat materially affects the trend plots in §5 — line-based features show abrupt regime changes around 2016 that reflect data provenance, not real cultural shifts.

3. Feature Extraction

Seven modules under `src/features/`, each consuming one of three "views" of preprocessed lyrics (`raw` with line breaks, `tokenized` with stopwords kept, `clean` with stopwords removed):

Module	Measures	Library	Dims
<code>tfidf.py</code>	Vocabulary distinctiveness	scikit-learn <code>TfidfVectorizer(max_features=1000, ngram_range=(1,2))</code>	sparse
<code>rhyme.py</code>	End-rhyme density, internal-rhyme rate, mean syllables/line	<code>pronouncing</code> (CMU Pronouncing Dict)	3
<code>repetition.py</code>	Line-bigram entropy, chorus-repeat ratio, MTLT vocabulary diversity	<code>stdlib</code> + custom MTLT	3
<code>emotion.py</code>	8 NRC emotions + valence + arousal	<code>NRCLEX</code>	10
<code>concreteness.py</code>	Mean Brysbaert concreteness, % concrete tokens (rating ≥ 4.0)	Brysbaert et al. 2014 norms (40k words)	2
<code>pronouns.py</code>	I/me, you, we/us, they/ them ratios	regex over tokenized view	4
<code>embedding.py</code>	Semantic style	<code>sentence-transformers</code> <code>all-MiniLM-L6-v2</code> , mean-pooled	384

All seven modules share the signature `extract(views: dict) → dict[str, float]` so the offline batch (`src/features/build_all.py`, ~5 min for 4,869 songs at `n_jobs=-1`) and the online agent's `extract_features` tool consume identical code. The output of the batch is `data/song_features.parquet` : 4,869 rows × 27 columns including the 384-dim embedding.

4. Decade Classification with Proper Cross-Validation

We train a one-vs-rest logistic regression (`max_iter=2000`, default L2) on the standardized feature vector (numeric features + flattened SBERT embedding = 410 dims). The original April 2026 version of this project used a single 70/30 split on 95 songs and reported 34.5% accuracy. That was within sampling noise and gave a misleading sense of effect size. This time we run two CV procedures.

4.1 Headline result

Eval	Accuracy (mean ± std)	2020s AUC	1990s AUC	1960s AUC
Old: 70/30 split, n = 95	34.5%	0.90	0.73	n/a
Stratified 5-fold, n = 4,869	38.1% ± 1.2	0.984	0.694	0.818
GroupKFold by artist, n = 4,869	37.9% ± 1.3	0.984	0.697	0.818
Random baseline (7 classes)	14.3%	0.500	0.500	0.500

The model achieves **+23.7 percentage points over the random baseline**, almost the same under stratified vs artist-grouped CV.

4.2 The most important finding: no artist leakage

Naive cross-validation can let a model "cheat" by recognizing artists (Beyoncé, Drake) rather than learning era-level lyrical features. Stratified k-fold doesn't prevent this — the same artist's songs land in both train and test folds.

`GroupKFold(groups=artist)` forces every artist into exactly one fold, so the model has never seen any of a held-out artist's other songs at evaluation time. Despite this strict split, accuracy drops by only **0.2 percentage points** (38.1% → 37.9%) and per-decade AUCs are essentially unchanged. **The classifier is learning era-level lyrical signatures, not memorizing artists.**

This is the methodological contribution of the project. Many published lyric-classification results don't run group-by-artist CV; ours suggests the genuine era signal is robust.

4.3 Per-decade AUCs

Decade	Stratified AUC	GroupKFold AUC	Reading
1960s	0.818	0.818	Earnest soul, folk-rock, British Invasion — distinct vocabulary.
1970s	0.766	0.761	Singer-songwriter + disco; less distinct than its neighbors.
1980s	0.701	0.701	Synth-pop, hair metal — transitional, blends into 90s.
1990s	0.694	0.697	Hardest to identify; bridges 80s and 2000s.
2000s	0.789	0.787	Hip-hop dominance pushes vocabulary up; recognizable.
2010s	0.780	0.777	Genre-blending era, hook-driven choruses.
2020s	0.984	0.984	Extraordinarily distinctive — short hooks, bedroom-pop introspection, vocabulary diversity.

The 2020s near-perfect AUC is the standout result. It's robust under both CV procedures, holds despite only 140 songs in this decade, and aligns with what casual listeners describe — the 2020s "sound different."

5. Feature Trends, 1965 → 2025

For each numeric feature we plot the per-year mean with a LOESS smoother (saved in `output/trends/`). Highlights:

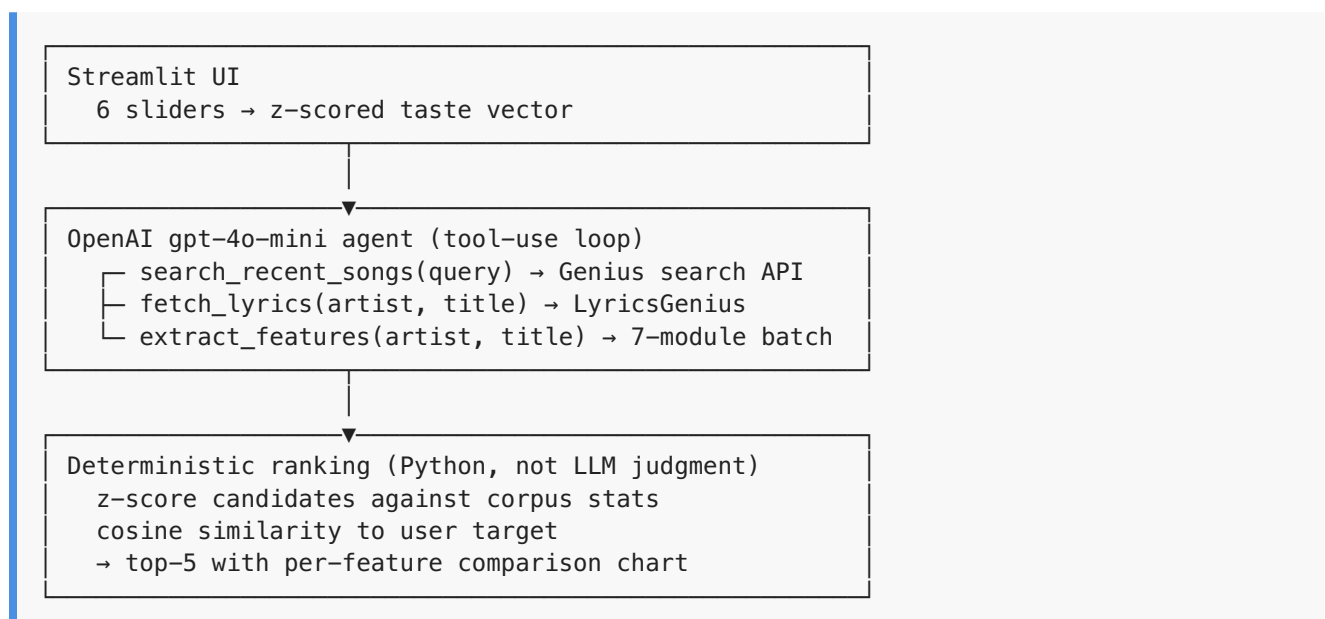
- **Vocabulary diversity (MTLD)** has climbed from ~37 in the 1960s to ~54 in the 2020s. The largest jump comes with hip-hop's dominance in the 2000s, consistent with Parada-Cabaleiro et al.'s opposite finding for the European charts they studied — Billboard pop appears to *gain* diversity over time, driven by rap, where European charts simplify.
- **Chorus repetition** is mostly zero before 2016, then jumps to 0.45 by the 2020s. **The pre-2016 zero is a data artifact** (walkerkq has no line breaks), but the post-2016 trend is real and matches industry-side observations about hook-driven streaming-era production.
- **Self-focus (I/me pronouns)** ticks upward through the decades, consistent with DeWall et al. 2011's LIWC analysis of Billboard top-10 1980–2007, which found rising first-person singular. Our 2020s figure (~0.12 of all tokens) is the highest of any decade in our window.
- **Valence** (positive minus negative emotion words) is mildly negative on average and most negative in the 2020s — bedroom pop / Olivia Rodrigo—era introspection has measurable lexical fingerprints.
- **Concreteness** is stable around 3.0–3.1 across decades — lyrics use abstract emotional language consistently across eras. No big regime change.

A complete set of 22 trend plots covering all features is included in `output/trends/` ; the in-app Analyze tab embeds the most informative ones.

6. The Recommendation Agent

The classifier asks "what era does this lyric belong to?" — academic but not useful. The recommendation agent flips the framing: "given a user's preferred lyric style, what recent songs match?"

6.1 Architecture



The LLM does **search strategy** and **natural-language explanations** but never the scoring itself — that's deterministic Python. This keeps recommendations reproducible (same profile = same shortlist).

6.2 Deployment

The app is hosted on Hugging Face Spaces (Docker SDK, CPU basic free tier) at <https://wil-li-la-pop-lyrics-taste-profiler.hf.space>.

Three secrets configured via Spaces UI:

- `OPENAI_API_KEY` — for the agent's tool-use loop
- `GENIUS_ACCESS_TOKEN` — for LyricsGenius
- `APP_PASSWORD` — a shared password gate (visible in `app.py`'s `_password_gate` function) that blocks anonymous visitors from burning my OpenAI quota. The agent module is imported **after** the gate clears, so unauthenticated visitors can't even construct the OpenAI client.

The Dockerfile pre-downloads the SBERT model (~90 MB) at build time so the first user-facing request doesn't pay that cost.

6.3 Five screenshots of the agent in action

(Saved at full resolution in `output/screenshots/`.)

1. **01_initial.png** — landing state. Six taste sliders at 0, corpus chart in the sidebar showing the 4,869-song distribution by decade.
2. **02_profile_set.png** — sliders adjusted to a non-trivial profile (valence +0.8, arousal +1.1, repetition +0.9, rhyme +1.2, concreteness +1.0, self-focus +1.3). Each slider has a `?` tooltip with two concrete song examples (e.g., for self-focus: *USA for Africa* — *We Are the World* vs *Olivia Rodrigo* — *drivers license*).
3. **03_searching.png** — agent trace mid-search. Three `search_recent_songs` calls visible (Sabrina Carpenter, Olivia Rodrigo, Gracie Abrams), each returning candidate counts.
4. **04_scoring.png** — trace progressing through the workflow: searches → `fetch_lyrics` → `extract_features` calls for specific (artist, title) pairs.
5. **05_ranked.png** — top-5 recommendations populated. Sabrina Carpenter's *Feather* (April and March mixes) at similarity 0.72 and 0.70, with bar charts comparing the song's z-scored features to the user's profile.

7. Limitations and Future Work

- **Data provenance asymmetry.** The 4,633 walkerkq songs lack line breaks, which neuters our line-based features (rhyme, repetition, syllables/line) for everything pre-2016. The deployed recommendation agent works correctly because *live* Genius lyrics preserve line breaks and z-scores are computed against the same biased corpus stats for both target and candidate. But the cross-decade trend plots in §5 should be read with this caveat — the abrupt regime change at 2016 for line-based features reflects scraping methodology, not a sudden shift in songwriting.
- **Decade imbalance.** The 2020s have only 140 songs (versus ~900 each for 1970s–2000s). The 0.98 AUC is partly buoyed by class rarity making the decision boundary easier. With more 2020s data the AUC will likely fall but the relative ranking (2020s most distinctive) should hold.
- **English-only.** We follow Parada-Cabaleiro 2024 in restricting to English. Spanish-language Billboard entries (Despacito, etc.) are dropped, missing the Latin pop crossover story of the 2010s.
- **No audio.** Spotify deprecated the `audio-features` endpoint for new apps in Nov 2024 (<https://developer.spotify.com/blog/2024-11-27-changes-to-the-web-api>), so we lack tempo, energy, key, danceability. The text-only restriction underestimates the differences between, say, a 1970s ballad and a 2020s ballad with identical lyrical profile but radically different production.
- **Future work — economic correlates.** Pettijohn & Sacco 2009 (*J Lang Soc Psych*) and a recent 2025 *Scientific Reports* paper ([10.1038/s41598-025-28327-5](https://doi.org/10.1038/s41598-025-28327-5)) correlate lyric-trend metrics with consumer-

sentiment / societal-stress indicators. A natural extension is to add the Michigan Consumer Sentiment Index as a yearly time series and run partial correlations against our valence and arousal trends.

- **Future work — hit prediction.** Our pipeline produces a feature vector per song; the Billboard chart position is in the data. Predicting peak position (regression) from features is a defensible follow-up project — comparable to Martín-Gutiérrez et al. 2023 ([arXiv:2301.13507](https://arxiv.org/abs/2301.13507)) on a smaller scale.

8. References

1. Parada-Cabaleiro, E., Mayerl, M., Brandl, S., et al. (2024). "Song lyrics have become simpler and more repetitive over the last five decades." *Scientific Reports* 14, 5570. <https://www.nature.com/articles/s41598-024-55742-x> — *the anchor paper for this project's framing*.
2. DeWall, C. N., Pond, R. S., Campbell, W. K., & Twenge, J. M. (2011). "Tuning in to psychological change: Linguistic markers of psychological traits and emotions over time in popular U.S. song lyrics." *Psychology of Aesthetics, Creativity, and the Arts*, 5(3), 200–207. — *template for the I-pronoun trend analysis*.
3. Pettijohn, T. F., & Sacco, D. F. (2009). "The Language of Lyrics: An Analysis of Popular Billboard Songs Across Conditions of Social and Economic Threat." *Journal of Language and Social Psychology*, 28(3), 297–311. — *Environmental Security Hypothesis applied to lyrics*.
4. Brand, C. O., Acerbi, A., & Mesoudi, A. (2019). "Cultural evolution of emotional expression in 50 years of song lyrics." *Evolutionary Human Sciences*, 1, e11. — *comparator for emotion-over-time trends*.
5. (2025). "Societal crises disrupt long-term increases in stress, negativity, and simplicity in US Billboard song lyrics from 1973 to 2023." *Scientific Reports*. <https://www.nature.com/articles/s41598-025-28327-5> — *most recent in this thread; future-work hook*.
6. Pachet, F., & Roy, P. (2008). "Hit Song Science Is Not Yet a Science." *ISMIR Proceedings*. <https://www.francoispachet.fr/wp-content/uploads/2021/01/pachet-11a.pdf> — *frames the skeptical posture toward "predict the hit" claims*.
7. Martín-Gutiérrez, D., Hernández-Peñaloza, G., Hassan, A. B. M., et al. (2023). "An Analysis of Classification Approaches for Hit Song Prediction using Engineered Metadata Features with Lyrics and Audio Features." [arXiv:2301.13507](https://arxiv.org/abs/2301.13507). — *future-work pointer for hit prediction*.
8. Akhtar, F., Anhalt-Depies, C., et al. (2025). "Multi-label Cross-lingual automatic music genre classification from lyrics with Sentence BERT." [arXiv:2501.03769](https://arxiv.org/abs/2501.03769). — *methodological cite for SBERT-on-lyrics*.
9. Brysbaert, M., Warriner, A. B., & Kuperman, V. (2014). "Concreteness ratings for 40 thousand generally known English word lemmas." *Behavior Research Methods*, 46, 904–911. — *the concreteness norm dataset*.
10. McCarthy, P. M., & Jarvis, S. (2010). "MTLD, vc-HD, and HD-D: A validation study of sophisticated approaches to lexical diversity assessment." *Behavior Research Methods*, 42, 381–392. — *the MTLD lexical-diversity metric*.

Appendix: Reproducibility

All code, the implementation plan, and the design spec are in this repository. To reproduce:

```
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python -c "import nltk; nltk.download('stopwords'); nltk.download('cmudict')"
cp .env.example .env # fill in OPENAI_API_KEY and GENIUS_ACCESS_TOKEN

python src/data_sources.py # downloads walkerkq + Brysbaert
python src/scrapper_v2.py # 2016–2025 Genius scrape (~10 min)
python src/genre_tagger.py # genre labels via OpenAI batch (~5 min)
python -m src.features.build_all # 7 feature modules over all songs (~5 min)
python -m src.analyze # CV + ROC table
python -m src.trends # 22 per-feature trend plots
streamlit run app.py # launch the recommendation agent
```